

P Y T H O N

自动化测试框架

——*Selenium*

Selenium最初是一个自动化测试工具，它可以驱动浏览器自动执行自定义好的逻辑代码，即通过代码完全模拟使用浏览器自动访问目标站点并操作，所以也因此应用于爬虫中。

01.基本用法

02.获取页面属性

03.定位页面元素

04.页面交互操作

05.多窗口切换

06.模拟鼠标键盘操作

07.延时等待操作

08.其他操作及实例

01

基本用法

熟悉初始化浏览器对象、访问页面、设置浏览器大小、刷新页面和前进后退等基础操作



(准备工作)：需要安装内容：Selenium框架、浏览器、驱动

1.1 初始化浏览器对象

```
from selenium import webdriver
import time
# 初始化浏览器为chrome浏览器
browser = webdriver.Chrome()
#等待5秒
time.sleep(5)

# 关闭浏览器
browser.close()
```

完成浏览器对象的初始化后并将其赋值给了**browser**对象，接下来我们就可以调用**browser**来执行各种方法模拟浏览器的操作了。

1.2 访问页面

```
from selenium import webdriver

# 初始化浏览器为chrome浏览器
browser = webdriver.Chrome()

# 访问百度首页
browser.get(r'https://www.baidu.com/')

# 关闭浏览器
browser.close()
```



进行页面访问使用的是**get**方法，传入参数为待访问页面的**URL**地址即可。

1.3 设置浏览器大小

```
from selenium import webdriver
import time

browser = webdriver.Chrome()

# 设置浏览器大小：全屏
browser.maximize_window()
browser.get(r'https://www.baidu.com')
time.sleep(2)

# 设置分辨率 500*500
browser.set_window_size(500,500)
time.sleep(2)

# 设置分辨率 1000*800
browser.set_window_size(1000,800)
time.sleep(2)

# 关闭浏览器
browser.close()
```

`set_window_size()`方法可以用来设置浏览器大小（就是分辨率），而`maximize_window`则是设置浏览器为全屏。

1.4 刷新页面

```
from selenium import webdriver
import time

browser = webdriver.Chrome()

# 设置浏览器全屏
browser.maximize_window()
browser.get(r'https://www.baidu.com')
time.sleep(2)
```

```
try:
    # 刷新页面
    browser.refresh()
    print('刷新页面')

except Exception as e:
    print('刷新失败')

# 关闭浏览器
browser.close()
```

刷新页面是我们在浏览器操作时很常用的操作，这里`refresh()`方法可以用来进行浏览器页面刷新。

1.5 前进后退

```
from selenium import webdriver  
import time
```

```
browser = webdriver.Chrome()
```

```
# 打开百度页面  
browser.get(r'https://www.baidu.com')  
time.sleep(2)
```

```
# 打开淘宝页面  
browser.get(r'https://www.taobao.com')  
time.sleep(2)
```

```
# 后退到百度页面  
browser.back()  
time.sleep(2)
```

```
# 前进的淘宝页面  
browser.forward()  
time.sleep(2)
```

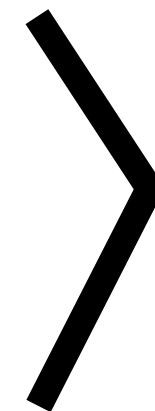
```
# 关闭浏览器  
browser.close()
```

前进后退也是我们在使用浏览器时非常常见的操作，这里 `forward()` 方法可以用来实现前进，`back()` 可以用来实现后退。

02

获取页面属性

当我们用selenium打开某个页面，有一些基础属性信息可以被直接获取。



获取页面属性

```
from selenium import webdriver
browser = webdriver.Chrome()
browser.get(r'https://www.baidu.com')
# 网页标题
print(browser.title)
# 当前网址
print(browser.current_url)
# 浏览器名称
print(browser.name)
# 网页源码
print(browser.page_source)
```

输出如下：

百度一下，你就知道

https://www.baidu.com/

chrome

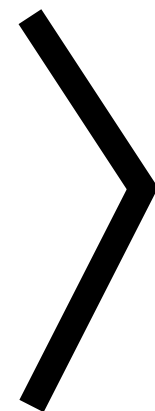
<html><head><script async="" src="... “

browser对象本身具有一些属性可供获取，平时常使用到的属性有网页标题，当前网址，浏览器名称和网页源码等，这些网页源码我们就可以通过**正则表达式**、**Bs4**、**xpath**以及**pyquery**等工具进行解析提取想要的信息了。

03

定位页面元素

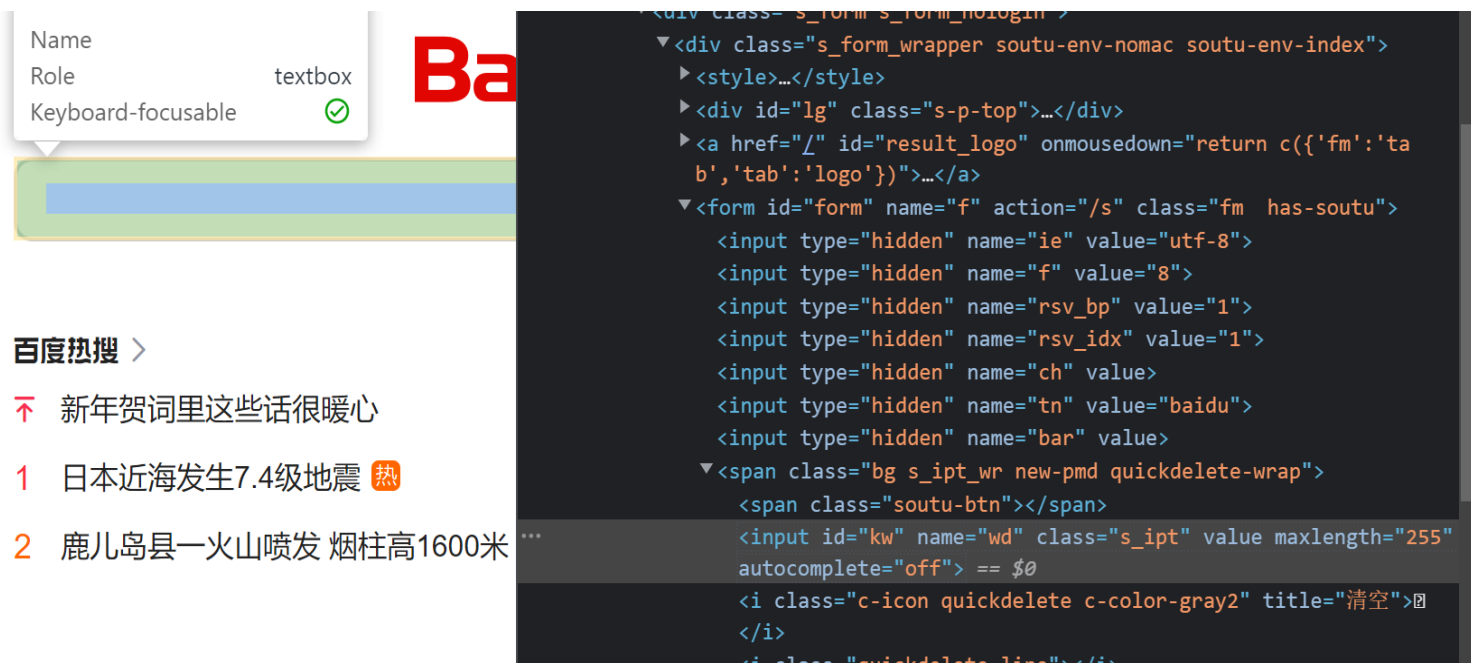
我们在实际使用浏览器的时候，很重要的操作有输入文本、点击确定等等。对此，Selenium提供了一系列的方法来方便我们实现以上操作。



3 find_element的By定位

常说的定位页面元素的操作方式一共有多种，下面选择几种常用方式进行介绍。

这里我们以百度首页的搜索框节点为例，在框中搜索“Python”这个词条。在搜索框->右键->检查



搜索框的html结构:

```
<input id="kw" name="wd" class="s_ipt" value="" maxlength="255" autocomplete="off">
```

```
<textarea id="chat-textarea" class="chat-input-textarea chat-input-scroll-style" data-ai-  
placeholder="" data-normal-placeholder="" rows="1" autocomplete="off">python</textarea>
```

3.1 id定位

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

browser = webdriver.Chrome()

browser.get(r'https://www.baidu.com')
time.sleep(2)

# 在搜索框输入 python
browser.find_element(By.ID, 'kw').send_keys('python')
time.sleep(2)

# 关闭浏览器
browser.close()
```

Chrome 正受到自动测试软件的控制。



- python什么东西
- python编程
- python怎么读
- python代码大全
- python培训机构哪家好
- python语言属于什么类型的语言
- python下载
- python和c++哪个更值得学
- python课程 网课推荐
- python编程软件

新版百度搜索框ID的值从'kw'改成了 'chat-textarea'

3.2 name和class定位



```
from selenium import webdriver
from selenium.webdriver.common.by
import By
import time

browser = webdriver.Chrome()

browser.get(r'https://www.baidu.com')
time.sleep(2)

# 在搜索框输入 python
browser.find_element(By.NAME,'w
d').send_keys('python')
time.sleep(2)
# 关闭浏览器
browser.close()
```

```
from selenium import webdriver
from selenium.webdriver.common.by
import By
import time

browser = webdriver.Chrome()

browser.get(r'https://www.baidu.com')
time.sleep(2)

# 在搜索框输入 python
browser.find_element(By.CLASS_NAME,'s_
ipt').send_keys('python')
time.sleep(2)
# 关闭浏览器
browser.close()
```

class的名称里面不能有空格，空格换成点即可。

3.3 tag定位

```
from selenium import webdriver
from selenium.webdriver.common.by
import By
import time

browser = webdriver.Chrome()

browser.get(r'https://www.baidu.com')
time.sleep(2)

# 在搜索框输入 python
browser.find_element
(By.TAG_NAME, 'input').send_keys('python')
time.sleep(2)

# 关闭浏览器
browser.close()
```

我们知道HTML是通过tag来定义功能的，比如input是输入，table是表格等等。

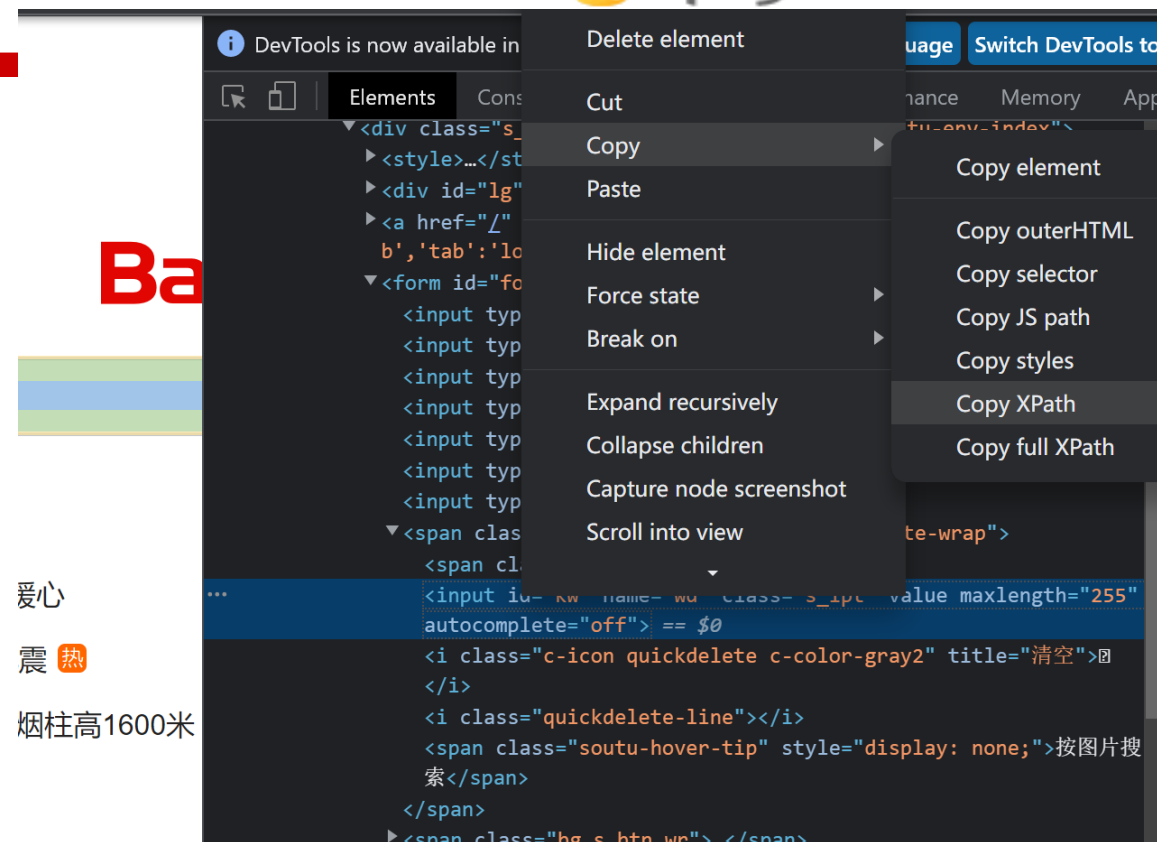
`find_element(By.TAG_NAME)`方法根据tag属性获取，我们查看百度首页的html代码，可以看到有很多同类tag，所以其实很难通过tag去区分不同的元素，因此由于该页面中存在多个input，

所以**左侧代码会报错**。

3.4 xpath定位

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

browser = webdriver.Chrome()
browser.get(r'https://www.baidu.com')
time.sleep(2)
# 在搜索框输入 python
browser.find_element(By.XPATH,'//*[@id="chat-textarea"]').send_keys('python')time.sleep(2)
# 关闭浏览器
browser.close()
```



前面介绍的几种定位方法是在理想状态下,在当前页面中,每个元素都有一个唯一的id或name或class的属性时可以使用。但是实际工作中并非这么美好,这时只能通过xpath或者css来定位了。

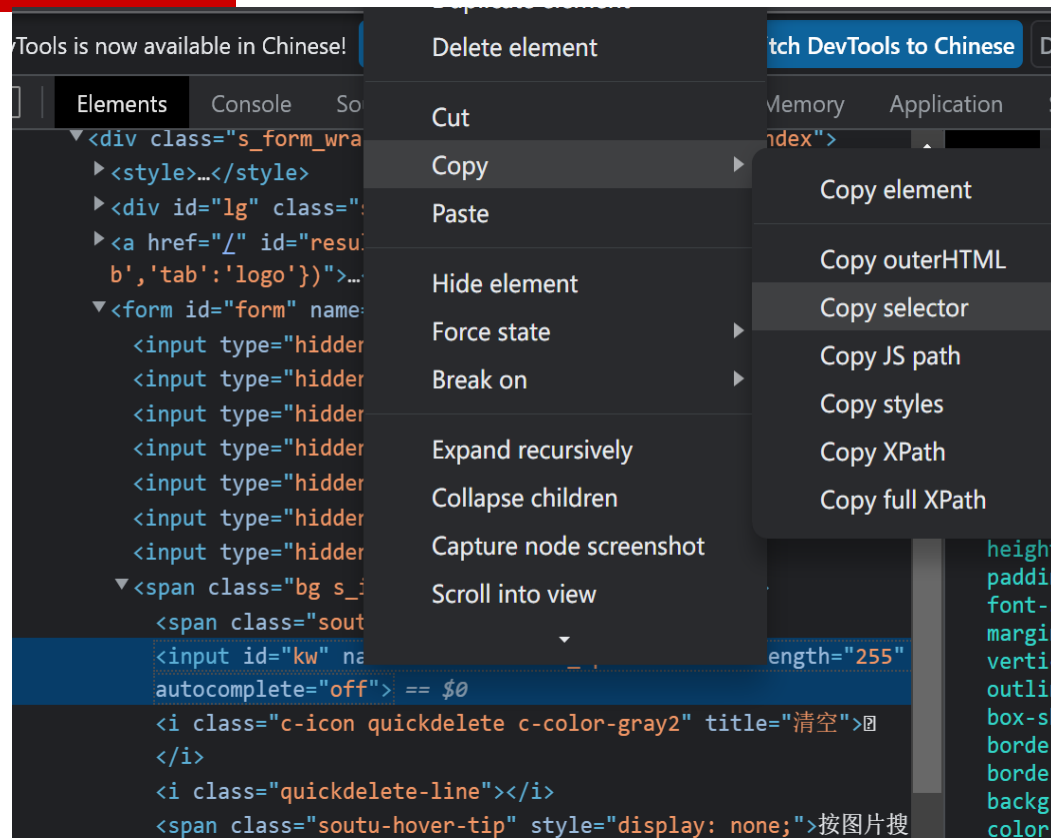
`find_element(By.XPATH)`方法用于xpath定位, xpath路径复制如上图。

3.5 css定位



```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
browser = webdriver.Chrome()
browser.get(r'https://www.baidu.com')
time.sleep(2)

# 在搜索框输入 python
browser.find_element(By.CSS_SELECTOR, '#chat-textarea').send_keys('python')
time.sleep(2)
# 关闭浏览器
browser.close()
```



3.6 link定位

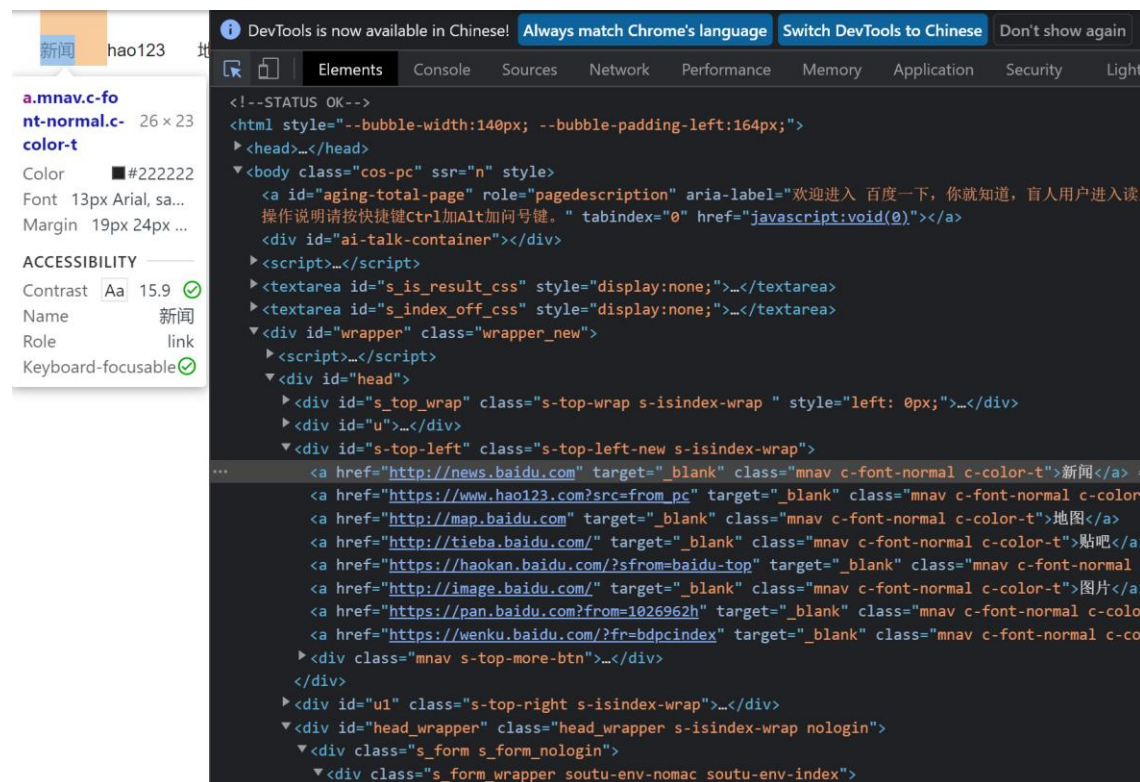


这种方法顾名思义就是用来定位文本链接的，以百度首页上方的分类模块链接为例，点击“新闻”超链接，该定位操作通过`find_element`方法实现。

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
browser = webdriver.Chrome()

browser.get(r'https://www.baidu.com')
time.sleep(2)

# 点击新闻 链接
browser.find_element(By.LINK_TEXT, '新闻').click()
time.sleep(2)
# 关闭浏览器全部页面
browser.quit()
```



3.7 partial定位



如果超链接的文本很长，如果全部输入既麻烦又显得代码不美观，这时候就可以只截取一部分字符串，用这种方法模糊匹配了。该定位操作可通过 `find_element` 的 `by.partial_link_text` 实现。

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

browser = webdriver.Chrome()
browser.get(r'https://www.baidu.com')
time.sleep(2)

# 点击新闻 链接
browser.find_element (By.PARTIAL_LINK_TEXT,'闻').click()
time.sleep(2)

# 关闭浏览器全部页面
browser.quit()
```

3.8 多个元素定位



如果定位的目标元素在网页中不止一个，那么则需要用到`find_elements`，得到的结果会是列表形式。简单来说，就是`element`后面多了复数标识`s`，其他操作一致。

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
browser = webdriver.Chrome()

browser.get(r'https://www.baidu.com')
time.sleep(2)

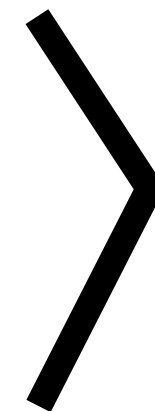
# 获取该页面所有"input"tag的元素
list_input = browser.find_elements(By.TAG_NAME, 'input')
time.sleep(2)

# 关闭浏览器
browser.close()
```

04

页面交互操作

页面交互就是在浏览器的各种操作，比如上面演示过的输入文本、点击链接等等，还有像清除文本、回车确认、单选框与多选框选中等。



4.1 输入文本操作

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
browser = webdriver.Chrome()
browser.get(r'https://www.baidu.com')
time.sleep(2)

# 定位搜索框
input = browser.find_element(By.CSS_SELECTOR, '#chat-textarea')

# 输入python
input.send_keys('python')
time.sleep(2)

# 关闭浏览器
browser.close()
```

定位到输入框位置，通过 **send_keys()** 方法输入文本“Python”。

4.2 点击操作

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
browser = webdriver.Chrome()
browser.get(r'https://www.baidu.com')
time.sleep(2)

# 选中新闻按钮
click = browser.find_element(By.LINK_TEXT,'新闻')
# 点击
click.click()
time.sleep(5)

# 关闭浏览器全部页面
browser.quit()
```

定位到“新闻”文本链接的位置，通过 **click()** 方法点击链接。

4.3 清除文本操作

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
browser = webdriver.Chrome()
browser.get(r'https://www.baidu.com')
time.sleep(2)
# 定位搜索框
input = browser.find_element(By.CLASS_NAME, 'chat-input-textarea.chat-
input-scroll-style')
# 输入python
input.send_keys('python')
time.sleep(2)
input.clear()      #清除
# 关闭浏览器
browser.close()
```

当输入的文本有误时，使用到`clear()`方法操作清除文本。

4.4 回车确认操作

```
from selenium import webdriver
from selenium.webdriver import Keys
from selenium.webdriver.common.by import By
import time
browser = webdriver.Chrome()
browser.get(r'https://www.baidu.com')
time.sleep(2)
input = browser.find_element(By.CLASS_NAME, 'chat-input-textarea.chat-
input-scroll-style')

input.send_keys('python')
time.sleep(2)
input.submit()      # 回车查询  input.send_keys(Keys.ENTER)
time.sleep(5)
# 关闭浏览器
browser.close()
```

在搜索框输入文本python，然后回车就直接弹出查询操作结果的情况，不用重复定位搜索按钮。

注：submit()只能用于包含属性type='submit'的标签，并且嵌套在form表单中。

4.5 下拉框的选择操作



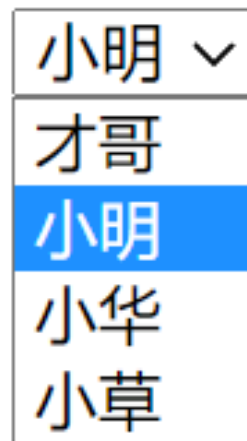
该操作需要用到select模块，在该模块中有以下三种定位方法

```
select_by_index()      # 通过索引定位；注意：index索引是从“0”开始。  
select_by_value()      # 通过value值定位，value标签的属性值。  
select_by_visible_text() # 通过文本值定位，即显示在下拉框的值。
```

以下图一个本地HTML为例：

```
<body>  
    
  <form>  
    <select name="人物">  
      <option value="才哥">才哥</option>  
      <option value="小明" selected="">小明</option>  
      <option value="小华">小华</option>  
      <option value="草儿">小草</option>  
    </select>  
  </form>
```

本地测试网页



网页运行

首先导入select类，然后再演示下拉框的不同选择的方式



```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.select import Select
import time
url = 'file:\\D:index.html'
browser = webdriver.Chrome()

browser.get(url)
time.sleep(2)
```

```
# 根据索引选择
Select(browser.find_element(By.NAME,"人
物")).select_by_index("2")
time.sleep(5)
# 根据value值选择
Select(browser.find_element(By.NAME,"人
物")).select_by_value("草儿")
time.sleep(5)
# 根据文本值选择
Select(browser.find_element(By.NAME,"人
物")).select_by_visible_text("王二妮")
time.sleep(5)
# 关闭浏览器
browser.close()
```

可以分别根据**索引值**，**value值**和**文本值**三种操作来进行下拉框选择。

05

多窗口切换

同一个页面的不同子页面的节点元素获取操作，不同选项卡之间的切换以及不同浏览器窗口之间的切换操作等



5.1 Frame切换



Selenium打开一个页面之后，默认是在父页面进行操作，如果这个页面还有子页面，想要获取子页面的节点元素信息则需要切换到子页面，这时`switch_to.frame()`就来了。如果想回到父页面，用`switch_to.parent_frame()`即可。

```
from selenium import webdriver
from selenium.webdriver.common.by
import By
import time
# 创建浏览器实例
browser = webdriver.Chrome()
# 打开父页面
browser.get('https://mail.qq.com')
# 切换到子页面
frame_element =
browser.find_element(By.TAG_NAME,'log
in_frame')
browser.switch_to.frame(frame_element)
time.sleep(2)
```

```
# 在子页面中进行操作
child_element =
browser.find_element(By.NAME,'phone')
child_element.click()
time.sleep(2)
# 切换回父页面
browser.switch_to.parent_frame()
# 在父页面中进行操作
parent_element =
browser.find_element(By.NAME,'q')
parent_element.click()

# 关闭浏览器实例
browser.quit()
```

5.2 选项卡切换



我们在访问网页的时候会打开很多个页面，在Selenium中提供了一些方法方便我们对这些页面进行操作。`current_window_handle`：获取当前窗口的句柄。`window_handles`：返回当前浏览器的所有窗口的句柄。`switch_to_window()`：用于切换到对应的窗口。

```
from selenium import webdriver
import time

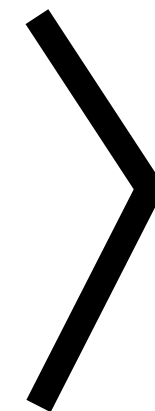
browser = webdriver.Chrome()
# 打开百度
browser.get('http://www.baidu.com')
# 新建一个选项卡
browser.execute_script('window.open()')
print(browser.window_handles)
time.sleep(2)
```

```
# 跳转到第二个选项卡并打开知乎
browser.switch_to.window(browser.window_handles[1])
browser.get('http://www.zhihu.com')
# 回到第一个选项卡并打开淘宝（原来的百度页面改为了淘宝）
time.sleep(2)
browser.switch_to.window(browser.window_handles[0])
browser.get('http://www.taobao.com')
```

06

模拟鼠标键盘操作

既然是模拟浏览器操作，自然也就需要能模拟鼠标和键盘的一些操作了。



6.1.1 鼠标左右键



当模拟这类操作时，需要先导入ActionChains 类。点击左键的实质与前面讲到的click()方法操作无异，而右键使用到的是context_click()方法。

```
from
selenium.webdriver.common.action_chains
import ActionChains
from selenium import webdriver
import time
from selenium.webdriver.common.by import
By
browser = webdriver.Chrome()
browser.get(r'https://www.baidu.com')
time.sleep(2)
```

```
# 定位到要右击的元素，这里选的新闻链接
right_click =
browser.find_element(By.LINK_TEXT,'新闻')
# 执行鼠标右键操作
ActionChains(browser).context_click(right_click).perform()
time.sleep(2)

browser.close()
```

ActionChains(browser): 调用ActionChains()类，并将浏览器驱动browser作为参数传入

context_click(right_click): 模拟鼠标双击，需要传入指定元素定位作为参数

perform(): 执行ActionChains()中储存的所有操作，可以看做是执行之前一系列的操作

6.1.2 双击



双击操作使用的是

`double_click()`方法。

```
from selenium.webdriver.common.action_chains import
ActionChains
from selenium import webdriver
import time
browser = webdriver.Chrome()
browser.get(r'https://www.baidu.com')
time.sleep(2)
# 定位到要双击的元素
double_click =
browser.find_element(By.CSS_SELECTOR,'#bottom_layer
> div > p:nth-child(8) > span')
# 双击
ActionChains(browser).double_click(double_click).perform()
time.sleep(2)
browser.close()
```

———

4级地震 热

山喷发 烟柱高1600米 热



```
> <p class="lh" style="display: none">
> <p class="lh" style="display: none">
> <p class="lh" style="display: none">
> <p class="lh" style="display: none">
  <div class="accessibility-icon">
...    <span class="c-icon">🔥</span> ==
    </div>
    <div class="open-content-info">...</div>
  </div>
</div>
<div id="bottom_space" class="s-bottom">
  <script type="application/json" id="p">
  </script>
</div>
> <div class="s_tab" id="s_tab">...</div>
> <div id="s_side_wrapper">...</div>
  <div id="wrapper_wrapper"></div>
</div>
<div class="c-tips-container" id="c-tips">
> <script>...</script>
  <script type="text/javascript" src="https://c/superman/js/lib/jquery-1-edb203c114.10">
  <script type="text/javascript" src="https://c/superman/js/lib/es1-d776bfb1aa.js"></script>
  <script data-for="es1-config">...</script>
```

6.1.3 拖拽



`drag_and_drop(source,target)`拖拽操作，开始位置和结束位置需要被指定，这个常用于滑块类验证码的操作之类。

```
from
selenium.webdriver.common.action_chains
import ActionChains
from selenium import webdriver
import time

browser = webdriver.Chrome()
url =
'https://www.runoob.com/try/try.php?file=
ame=jqueryui-api-droppable'
browser.get(url)
time.sleep(2)

browser.switch_to.frame('iframeResult')
```

```
# 开始位置
source = browser.find_element(By.CSS_SELECTOR,
"#draggable")
# 结束位置
target =
browser.find_element(By.CSS_SELECTOR,"#droppa
ble")
# 执行元素的拖放操作
actions = ActionChains(browser)
actions.drag_and_drop(source, target)
actions.perform()
# 拖拽
time.sleep(15)
browser.close()
```

6.1.4 悬停

```
from selenium.webdriver.common.action_chains import
ActionChains
from selenium import webdriver
import time
browser = webdriver.Chrome()
url = 'https://www.baidu.com'
browser.get(url)
time.sleep(2)
# 定位悬停的位置
move = browser.find_element(By.CSS_SELECTOR,"#form >
span.bg.s_ipt_wr.new-pmd.quickdelete-wrap > span.soutu-btn")

# 悬停操作
ActionChains(browser).move_to_element(move).perform()
time.sleep(5)
# 关闭浏览器
browser.close()
```



悬停操作使用到的是
`move_to_element()`方法。



6.2 键盘操作



```
from selenium.webdriver.common.keys import Keys
from selenium import webdriver
import time
from selenium.webdriver.common.by import By
browser = webdriver.Chrome()
url = 'https://www.baidu.com'
browser.get(url)
time.sleep(2)
# 定位搜索框
input =
browser.find_element(By.CLASS_NAME,'? ? ? ')
# 输入python
input.send_keys('python')
time.sleep(2)
# 回车
input.send_keys(Keys.ENTER)
time.sleep(5)
browser.close()
```

selenium中的Keys()类提供了大部分的键盘操作方法，通过send_keys()方法来模拟键盘上的按键。定位到需要操作的元素进行操作即可。

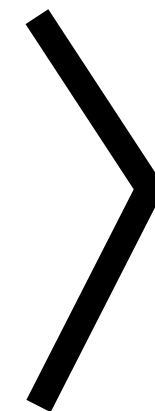
常见键盘操作：

send_keys(Keys.BACK_SPACE): 删除键(BackSpace)
send_keys(Keys.SPACE): 空格键(Space)
send_keys(Keys.TAB): 制表键(TAB)
send_keys(Keys.ESCAPE): 回退键(ESCAPE)
send_keys(Keys.ENTER): 回车键(ENTER)
send_keys(Keys.CONTROL,'a'): 全选(Ctrl+A)
send_keys(Keys.CONTROL,'c'): 复制(Ctrl+C)
send_keys(Keys.CONTROL,'x'): 剪切(Ctrl+X)
send_keys(Keys.CONTROL,'v'): 粘贴(Ctrl+V)
send_keys(Keys.F1): 键盘F1
.....
send_keys(Keys.F12): 键盘F12

07

延时等待操作

如果遇到使用ajax加载的网页，页面元素可能不是同时加载出来的，这种情况下需要设置延时等待一定时间，确保全部节点都加载出来。



7.1 强制等待



强制等待是最简单的一种，直接用`time.sleep(n)`强制等待n秒，在执行`get`方法之后执行。

7.2 隐式等待

`implicitly_wait()`设置等待时间，如果到时间有元素节点没有加载出来，就会抛出异常。

```
from selenium import webdriver

browser = webdriver.Chrome()

# 隐式等待，等待时间10秒
browser.implicitly_wait(10)

browser.get('https://www.baidu.com')
print(browser.current_url)
print(browser.title)

# 关闭浏览器
browser.close()
```

7.3 显式等待



设置一个等待时间和一个条件，在规定时间内，每隔一段时间查看下条件是否成立，如果成立那么程序就继续执行，否则就抛出一个超时异常。

```
from selenium import webdriver
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By
import time

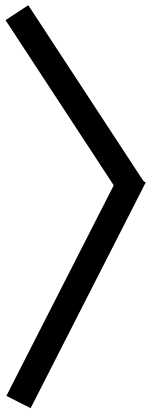
browser = webdriver.Chrome()
browser.get('https://www.baidu.com')
# 设置等待时间10s
wait = WebDriverWait(browser, 10)
# 设置判断条件：等待id='chat-textarea'的元素加载完成
input = wait.until(EC.presence_of_element_located((By.ID,'chat-textarea')))
# 在关键词输入：关键词
input.send_keys('Python')

time.sleep(2)
browser.close()
```

08

其他操作

补充一些其他用法



8.1 运行JavaScript



还有一些操作，比如下拉进度条，模拟JavaScript，使用execute_script方法来实现。

```
from selenium import webdriver

browser = webdriver.Chrome()
# 知乎发现页
browser.get('https://www.zhihu.com/explore')

browser.execute_script('window.scrollTo(0, document.body.scrollHeight)')
browser.execute_script('alert("To Bottom")')
```



案例：用js对12306日期修改

12306首页中的日期输入框修改，而**selenium**中没有直接的方法可以修改元素的属性，只有通过**js**才能做到，那么接下来我们就使用**execute_script**方法来执行对应的**js**代码。

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.wait import WebDriverWait
import time
driver = webdriver.Chrome()
driver.implicitly_wait(30)
driver.get('https://www.12306.cn/index/')
# 点击往返
WebDriverWait(driver, 10, 0.5).until(
    EC.element_to_be_clickable(
        (By.XPATH, '//div[@class="search-tab-hd"]/a[text()="往返"]))).click()

# 定位起始时间和终止时间输入框
start_date = driver.find_element(By.ID, 'go_date')
end_date = driver.find_element(By.ID, 'from_date')
# 准备js代码，通过js输入起始时间和终止时间，
js = """
var s_ele = arguments[0];
var e_ele = arguments[1];
s_ele.readonly = false;
s_ele.value = '2023-05-15';
e_ele.readonly=false;
e_ele.value = '2023-06-15';
return [s_ele.value,e_ele.value]
"""
# js中的arguments[0]接收的是args中的第1个参数，就是下面传入的start_date
# js中的arguments[1]接收的是args中的第2个参数，就是下面传入的end_date
# js代码中可以通过return来返回js代码执行之后的结果
# 执行js代码
res = driver.execute_script(js, start_date, end_date)
time.sleep(15)
driver.quit()
```

8.2 利用lelenium将数据传到前端



方法1、前端设置1个隐藏元素demo，用js脚本把json传过去

```
driver.execute_script("return document.getElementById('demo').value='"+data+"'");")
```

方法2、通过id定位元素demo，然后sendKeys发送值

```
element=driver.find_element(By.ID,'demo')  
Element.sendKeys(data)
```

8.3 Cookie



在selenium使用过程中，还可以很方便对Cookie进行获取、添加与删除等操作。

```
from selenium import webdriver
browser = webdriver.Chrome()
# 知乎发现页
browser.get('https://www.zhihu.com/explore')
# 获取cookie
print(f'Cookies的值: {browser.get_cookies()}')
# 添加cookie
browser.add_cookie({'name': '才哥', 'value': '人物'})
print(f'添加后Cookies的值: {browser.get_cookies()}')
# 删除cookie
browser.delete_all_cookies()
print(f'删除后Cookies的值: {browser.get_cookies()}')
```

```
Cookies的值: [{'domain': '.zhihu.com', 'httpOnly': False, 'name': 'Hm_lpv', 'value': '1234567890', 'path': '/'}]
添加后Cookies的值: [{'domain': 'www.zhihu.com', 'httpOnly': False, 'name': '才哥', 'value': '人物', 'path': '/'}]
删除后Cookies的值: []
```

09

爬取动态页面实例



爬取2023软科中国大学排名网站:

<https://www.shanghairanking.cn/rankings/bcur/202311>



SHANGHAI RANKING 排名 院校 报告 资讯 English 登录/注册

排名 > 中国大学排名

2023中国大学排名

“软科中国大学排名”前身是“中国最好大学排名”，自2015年首次发布以来，以专业、客观、透明的优势赢得了高等教育领域内外的广泛关注和认可，已经成为具有重要社会影响力和权威参考价值的中国大学排名领先品牌。软科中国大学排名以服务中国高等教育发展和进步为导向

展开全部

中国大学排名 (主榜) 共 590 所学校

请输入学校名称

排名	学校名称	省市	类型	总分	办学层次
1	清华大学 Tsinghua University 双一流/985/211	北京	综合	1004.1	37.5
2	北京大学 Peking University 双一流/985/211	北京	综合	910.5	35.0
3	浙江大学 Zhejiang University 双一流/985/211	浙江	综合	822.9	35.9
4	上海交通大学 Shanghai Jiao Tong University 双一流/985/211	上海	综合	778.6	35.7
5	复旦大学 Fudan University 双一流/985/211	上海	综合	712.4	36.5

中国大学排名 (主榜)

- 中国医药类大学排名
- 中国财经类大学排名
- 中国语言类大学排名
- 中国政法类大学排名
- 中国民族类大学排名
- 中国体育类大学排名
- 中国合作办学大学排名
- 中国大学排名 (总榜)
- 中国民办高校排名 (主榜)
- 中国民办财经类高校排名

查看更多

按照“科学水平”这一筛选条件，将“中国大学排名（主榜）”中“北京”地区的“综合”类高校的名字及其对应的分数按排名顺序爬取下来。

按照“科学水平”，将“中国大学排名（主榜）”中“北京”地区的“综合”类高校的名字及分数按排名顺序爬取下来。



导入Selenium库中的webdriver模块和By模块，它定义了一些常用的元素查找方式。

导入Python标准库中的time模块，以便在需要时添加等待时间。

创建一个Chrome浏览器实例，使用get()方法打开要爬取的目标网页。

```
from selenium import webdriver
import time
from selenium.webdriver.common.by import By

url = 'https://www.shanghairanking.cn/rankings/bcur/202311'
//初始化浏览器并传入网址
browser = webdriver.Chrome()
browser.maximize_window()
browser.get(url)
```

通过find_element()方法中的CSS选择器定位到“办学层次”下拉框元素并单击。



```
<th style="width:60px" data-v-4645600d class> 排名</th>
<th class="align-left" style="width:auto" data-v-4645600d> 学校名称
</th>
▶<th style="width:80px" data-v-4645600d class>⋮</th>
▶<th style="width:80px" data-v-4645600d class>⋮</th>
▶<th style="width:80px" data-v-4645600d class>⋮</th>
▼<th style="width:170px" data-v-4645600d class>
  ▼<div class="rk-sort-slot" data-v-4645600d> flex
    ▼<div class="rank-select" style="width:139px!important;height:30px"
      data-v-11b0cbc6 data-v-a2361c36 data-v-4645600d>
        ▼<div class="inputWrapper" style="font-size:18px" data-v-
          11b0cbc6>
          <input readonly="readonly" placeholder="选择" value="办学层次"
            class="head-bg" data-v-11b0cbc6 type="text"> == $0
          
          </div>
          ▶<div class="rk-tooltip" style="position: absolute; inset: 0px au
            to auto 0px; margin: 0px; transform: translate(864px, 426px); di
            splay: none;" data-v-11b0cbc6 data-popover-placement="bottom-star
            t">⋮</div>
```

python™

请输入学校名称

省市	类型	总分	办学层次
北京	综合	1004.1	37.5
北京	综合	910.5	35.0
浙江	综合	822.9	35.9

```
##定位到办学层次下拉框并单击
browser.find_element(by=By.CSS_SELECTOR,
                    value=f'#content-box > div.rk-table-box > table > thead > tr > th:nth-child(6)'
                    f' > div > div.rank-select > div.inputWrapper > input').click()
```

再以同样方式定位到“科学水平”选项元素并单击。

```
<th style="width:80px" data-v-4645600d class></th>
<th style="width:170px" data-v-4645600d class>
  <div class="rk-sort-slot" data-v-4645600d> flex
    <div class="rank-select" style="width:139px!important;height:30px" data-v-11b0cbc6 data-v-a2361c36 data-v-4645600d>
      <div class="inputWrapper" style="font-size:18px" data-v-11b0cbc6>
      <div class="rk-tooltip" style="position: absolute; inset: 0px auto auto 0px; margin: 0px; transform: translate(864px, 121px);" data-v-11b0cbc6 data-popper-placement="bottom-start">
        <ul class="options" style="min-width:139px!important" data-v-11b0cbc6>
          <li class="select-active" data-v-11b0cbc6>办学层次</li>
          <li data-v-11b0cbc6 class>学科水平</li> == $0
          <li data-v-11b0cbc6 class>办学资源</li>
          <li data-v-11b0cbc6 class>师资规模与结构</li>
          <li data-v-11b0cbc6 class>人才培养</li>
          <li data-v-11b0cbc6 class>科学研究</li>
          <li data-v-11b0cbc6 class>服务社会</li>
          <li data-v-11b0cbc6 class>高端人才</li>
          <li data-v-11b0cbc6 class>重大项目与成果</li>
          <li data-v-11b0cbc6 class>国际竞争力</li>
        </ul>
      </div>
    </div>
  </div>
```



##选择学科水平选项

```
browser.find_element(by=By.CSS_SELECTOR,
                    value=f'#content-box > div.rk-table-box > table > thead > tr > th:nth-child(6)'
                    f' > div > div.rank-select > div.rk-tooltip > ul > li:nth-child(2)').click()
```

以上述同样的方式定位到“类型”下拉框并选择“综合”选项



```
<th style="width:80px" data-v-4645600d class>⋮</th>
▼<th style="width:80px" data-v-4645600d class>
  ▼<div class="rk-sort-slot" data-v-4645600d> flex
    ▼<div class="rank-select" style="width:61px!important; height:30px" data-v-11b0cbc6 data-v-a2361c36 data-v-4645600d>
      ▼<div class="inputWrapper" style="font-size:18px" data-v-11b0cbc6>
        <input readonly="readonly" placeholder="选择" value="全部" class="head-bg" data-v-11b0cbc6 type="text"> == $0
        
      </div>
```

```
t: 0px auto auto 0px; margin: 0px; transform: translate(708px, 121px);" data-v-11b0cbc6 data-popover-placement="bottom-start">
  ▼<ul class="options" style="min-width:61px!important" data-v-11b0cbc6>
    <li class="select-active" data-v-11b0cbc6>全部</li>
    <li data-v-11b0cbc6 class>综合</li> == $0
    <li data-v-11b0cbc6 class>理工</li>
    <li data-v-11b0cbc6 class>师范</li>
    <li data-v-11b0cbc6 class>农业</li>
    <li data-v-11b0cbc6 class>林业</li>
  </ul>
</div>
```

##定位类型下拉框并点击

```
browser.find_element(by=By.CSS_SELECTOR,
                      value=f'#content-box > div.rk-table-box > table > '
                              f'thead > tr > th:nth-child(3) > div > div'
                              '> ')
browser.find_element(by=By.CSS_SELECTOR,
                      value=f'#content-box > div.rk-table-box > table > '
                              f'thead > tr > th:nth-child(3) > div > div'
                              '> ')
```

```
browser.find_element(by=By.CSS_SELECTOR,
                      value=f'#content-box > div.rk-table-box > table > '
                              f'thead > tr > th:nth-child(3) > div > div'
                              '> ')
```

##定位综合选项并点击

```
browser.find_element(by=By.CSS_SELECTOR,
                      value=f'#content-box > div.rk-table-box > table > '
                              f'thead > tr > th:nth-child(3) > div > div'
                              '> ')
```



以上述同样的方式定位到“省市”下拉框并分别选择“北京”选项



```
</th>
▼ <th style="width:80px" data-v-4645600d class>
  ▼ <div class="rk-sort-slot" data-v-4645600d> flex
    ▼ <div class="rank-select" style="width:61px!important;height:30px" data-v-11b0cbc6 data-v-a2361c36 data-v-4645600d>
      ▼ <div class="inputWrapper" style="font-size:18px" data-v-11b0cbc6>
        <input readonly="readonly" placeholder="选择" value="全部" class="head-bg" data-v-11b0cbc6 type="text"> == $0
        
      </div>
```

```
</div>
▼ <div class="rk-tooltip" style="position: absolute; inset: 0px auto auto 0px; margin: 0px; transform: translate(628px, 121px);" data-v-11b0cbc6 data-popper-placement="bottom-start">
  ▼ <ul class="options" style="min-width:61px!important" data-v-11b0cbc6>
    <li class="select-active" data-v-11b0cbc6>全部</li>
    <li data-v-11b0cbc6 class>安徽</li>
    <li data-v-11b0cbc6 class>北京</li> == $0
    <li data-v-11b0cbc6 class>重庆</li>
    <li data-v-11b0cbc6 class>福建</li>
    <li data-v-11b0cbc6 class>甘肃</li>
```

##定位省市下拉框并点击

```
browser.find_element(by=By.CSS_SELECTOR,
                      value=f'#content-box > div.rk-table-box >
table > '
                      f'thead > tr > th:nth-child(3) > div > div
> '
                      f'div.inputWrapper > input').click()
```

##定位北京选项框并点击

```
browser.find_element(by=By.CSS_SELECTOR,
                      value=f'#content-box > div.rk-table-box >
table > '
                      f'thead > tr > th:nth-child(3) > div > div
```

省市	类型	总分	办学层次
全部			
安徽	综合	1004.1	
北京			
重庆			
福建			
甘肃	综合	910.5	
广东			
广西			
贵州	综合	822.9	
海南			
河北			

至此，上述步骤已将需要爬取的页面加载完毕。

接下来通过`find_elements_by_xpath()`方法来查找并获取多个匹配某个XPath表

达式的元素，即学校的名字及对应分数，分别放入列表。



```
##校名
elements = browser.find_elements_by_xpath(
    '//*[@id="content-box"]/div[2]/table/tbody//tr/td[2]/div/div[2]/div[1]/div/div/a')
```

```
list_name = []
for element in elements:
    list_name.append(element.text)
```

```
##总分
elements2 = browser.find_elements_by_xpath(
    '//*[@id="content-box"]/div[2]/table/tbody//tr/td[5]')
```

```
list_score = []
for element2 in elements2:
    list_score.append(element2.text)
```

中国大学排名（主榜） 共 5 所学校

请输入学校名称

排名	学校名称	北京	综合	总分	学科水平
1	 清华大学 Tsinghua University 双一流/985/211	北京	综合	1004.1	77.1
2	 北京大学 Peking University 双一流/985/211	北京	综合	910.5	74.3
18	 中国人民大学 Renmin University of China 双一流/985/211	北京	综合	526.3	48.9
202	 国际关系学院 University of International ...	北京	综合	169.9	
276	 北京联合大学 Beijing Union University	北京	综合	140.6	

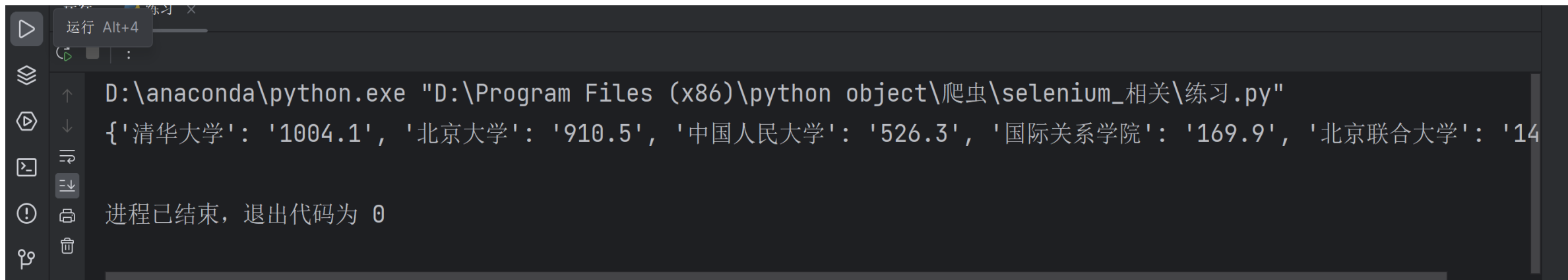
* 排名或排名区间相同的大学按校名拼音首字母顺序排列（总榜中主榜院校在前）

将爬取到的校名列表和分数列表合并为字典并返回。



```
result = {k: v for k, v in zip(list_name, list_score)}  
print(result)
```

爬取结果结果如下图，即成功获取到了对应要求的校名及分数。

A screenshot of a terminal window with a dark background. The window title is "练习". The command prompt shows the execution of a Python script: "D:\anaconda\python.exe "D:\Program Files (x86)\python object\爬虫\selenium_相关\练习.py"". The output is a dictionary: {'清华大学': '1004.1', '北京大学': '910.5', '中国人民大学': '526.3', '国际关系学院': '169.9', '北京联合大学': '14...'. Below the output, it says "进程已结束，退出代码为 0". The terminal has a sidebar on the left with various icons for file management and debugging.

```
D:\anaconda\python.exe "D:\Program Files (x86)\python object\爬虫\selenium_相关\练习.py"  
{'清华大学': '1004.1', '北京大学': '910.5', '中国人民大学': '526.3', '国际关系学院': '169.9', '北京联合大学': '14...  
进程已结束，退出代码为 0
```